AERSP 304 – Dynamics and Controls of Aerospace Systems

Funduino Report 1

DC Motor System Analysis

Luke Clemens, Robert Danner, and Joseph Parker

Introduction

System identification is the process of building a mathematical model of a dynamic system using input and output data rather than being based on pure theory. By applying a known input signal and observing the system response, model parameters can be estimated from real behavior.

Particularly, this report documents the system identification of a 12V DC geared motor. The motor is powered and driven by an Arduino Mega 2560 R3 through an L298N H-bridge. Code is written in Arduino IDE and sent through the Arduino to power the motor. Step voltage inputs at five different amplitudes were applied to the motor, and the resulting RPM response was recorded to identify a first-order transfer function. The transfer function was of the form $G(s) = K/(\tau s+1)$, where K is the DC gain, and τ is the time constant. A chirp signal was then used to further validate the identified model across a variety of frequencies.

Mathematical Derivations

Background

The electrical motor converts electrical energy to mechanical energy in the form of torque, represented by the motor spinning. When a voltage is applied, a current develops and, in turn, accelerates the motor. The following equations and model can be used to accurately describe the mechanics of this system:

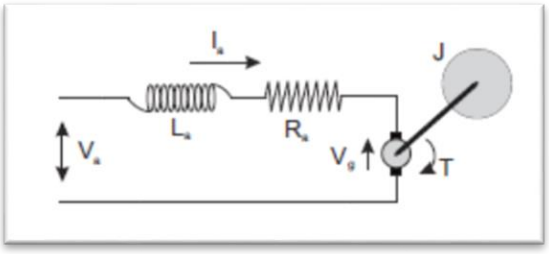


Figure 1: Schematics for the motor system

Nomenclature:

Input Voltage	V_a

Armature Current	i_a
Armature Inductance	L_a
Armature Resistance	R_a
Back-EMF	V_g
Motor Torque Constant	K_m
Back-EMF Constant	K_g
Motor Torque	T
Rotor Moment of Inertia	J
Angular Velocity	ω
Angular Position	θ

Electrical Equations

$$L_a \frac{di_a}{dt} = V_a - R_a \cdot i_a - V_g$$

$$T = K_m \cdot i_a.$$

Mechanical Equations

$$V_g = K_g \cdot \omega.$$

$$\omega(t) = \frac{d\theta}{dt} = \dot{\theta}(t).$$

$$J\frac{d\omega}{dt} = K_m \cdot i_a$$

Transfer Function Derivation

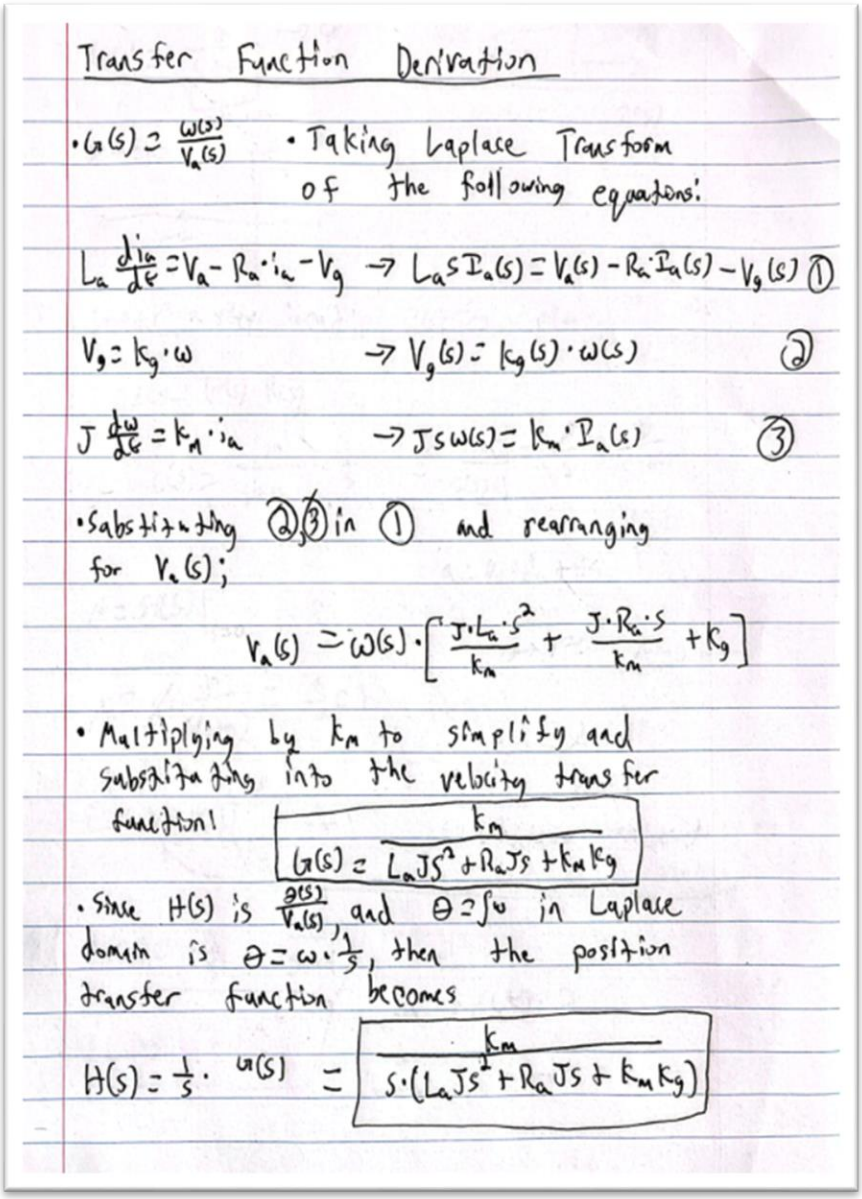


Figure 2: Transfer Function Derivation by Hand

The transfer function for G(s) is a Type 0 system since there is no pole at the origin. Meanwhile H(s) is a Type 1 system since there does exist a pole at the origin.

Hardware Setup

The first task was to get the motor to spin using the Arduino and h-bridge. First, a USB-A to USB-B cable was used to power the Arduino through the computer supplying the code. The following is a diagram of the L298N h-bridge and Arduino board given by Amazon to use as a reference:

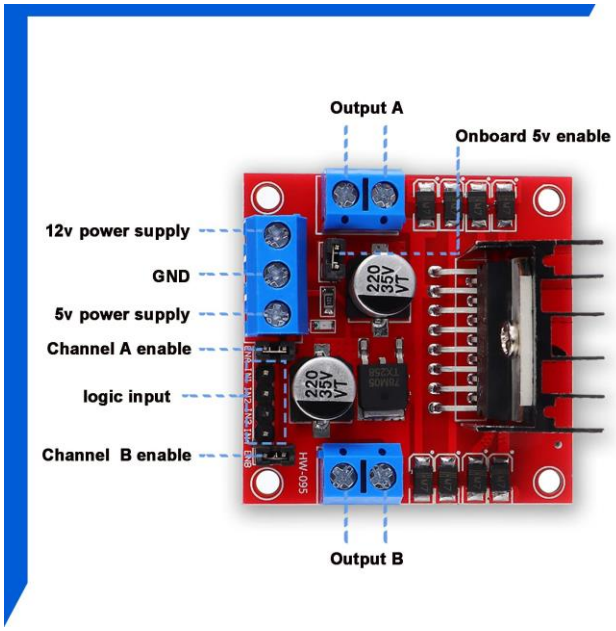


Figure 3: H-Bridge L298N Diagram



Figure 4: Arduino Mega 2560 R3

To connect to the Arduino, a black wire connected the ENA spot on the L298N to Digital 9 on the Arduino. A red wire was used to connect the IN1 spot to Digital 7 on Arduino, and a black wire connected IN2 to Digital 8. Now, all that needed to be done was to connect the 12V power adapter to the h-bridge to generate enough volts to power the motor, which would also be connected to the h-bridge. A wire was connected from the negative terminal to the ground terminal on the h-bridge, and another wire was connected from the positive

terminal to the 12V input on the L298N. A jumper cap was also placed on the “Onboard 5V Enable” to run through the regulator and create its own 5V to power its chip’s logic. Finally, the positive wire from the motor (white) is connected to output 1 in the Output A section, and the negative wire from the motor (black) is connected to Output 2 in output A as shown in the following circuit:

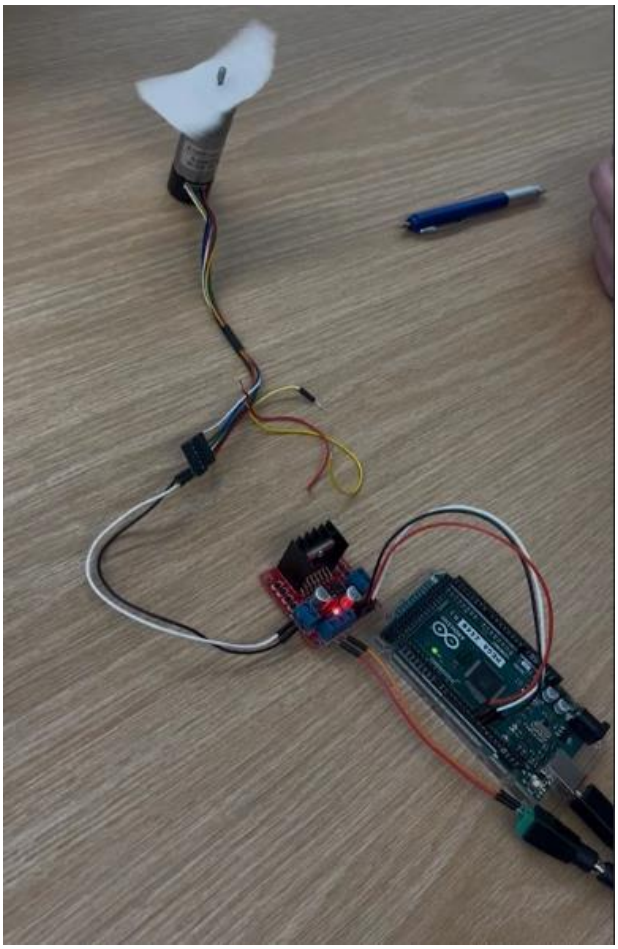


Figure 5: Circuit to spin motor

After the motor successfully spun, the encoders needed to be connected to record the data being presented by the motor and Arduino. The encoder power supply (blue), already connected to the motor, connected to the 5V slot on the Arduino power for power. Then, for grounding, the green encoder grounding wire was connected to the ground slot on the Arduino board. Then, Encoders A and B (yellow and white) were connected to PWM 2 and 3 on the Arduino board. The following figures are images of the completed circuit:

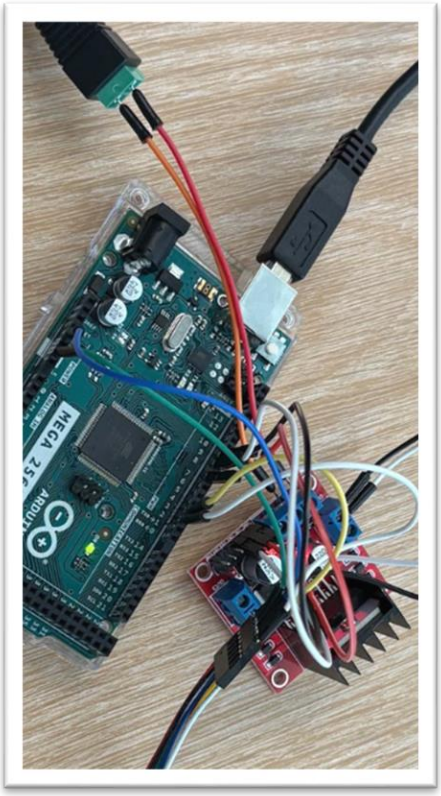


Figure 6: Zoomed in Circuit

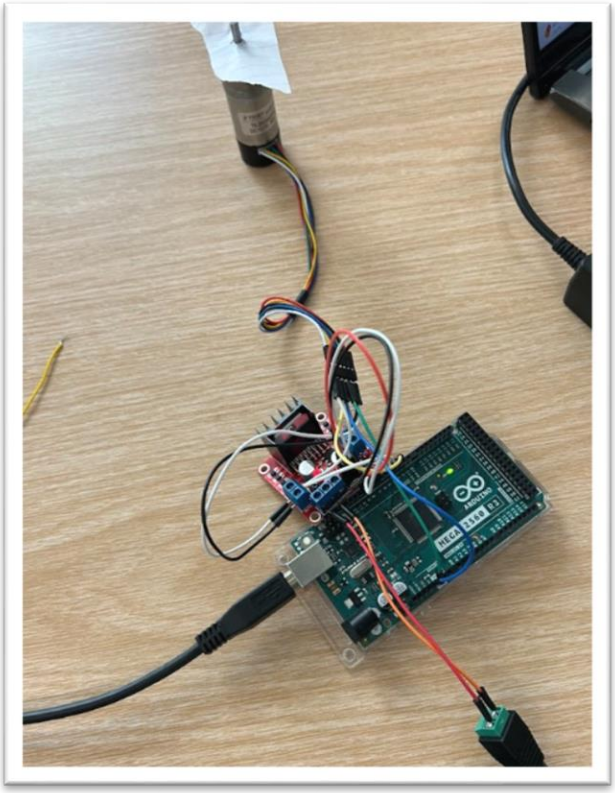


Figure 7: Completed Circuit

Reasoning and Logic to Build Code

The code was built in Arduino IDE to produce the signal. It includes brief comments and assigns names to each pin to simplify the code and make it more understandable to begin with. The constants used in the code come from the specifications of the 300 RPM motor listed in the "Funduino Getting Started" canvas module. The supply voltage is what the required input was with a sample rate every .1 seconds.

The encoderISR functions fire automatically each time the encoder generates a pulse incrementing the encoderCount variable. The CPR is the total number of pulses per full output shaft revolution. It is determined by the encoder hardware and counting method. The specifications presented a 12 pulse per revolution rate with a gear ratio of 20:1. Due to having 2 encoder channels present in the motor wiring and having a change mode capturing rising and falling edges involved, the multiplication of those numbers yielded a counts-per-revolution-rate of 960.

Based on the wiring of the H-Bridge to the Arduino, the IN1 and IN2 are used to control the motor. The setMotorPWM function then sends a signal to the H-Bridge to regulate the voltage being supplied to the motor. The code has a built-in RPM counter from the encoder which counts to provide data, and a built-in PWM-to-voltage function to convert that signal to an equivalent

voltage. Between each trial, the stopmotor function cuts the power to the motor to allow for non-corrupted results.

Step trials are used in this experiment through the runStepTrial. It resets the encoder counts and takes measurements at the assigned sampling rate for 12 seconds. To ensure data integrity, built-in functions are used to prevent modifications to the encoder while the data is being read. This is done to keep the data precise and accurate to what is being relayed from the encoder to the Arduino through the nointerrupts, and interrupts code within the function. The data at each sample is recorded in the serial monitor to later be used for analysis. This is looped for five test trials decreasing the voltage from 80% to 40% of the maximum voltage.

The chirp trials run in a similar way to the step trials with a caveat of it starting at a low frequency of .1Hz and increasing to 2.0Hz over 20 seconds with the voltage output being varied by equation 1.

$$V_a(t) = \frac{A}{2} + \frac{A}{2} * \sin\left(2\pi\left(\frac{c}{2} * t^2 * f_o * t\right)\right) \quad (1)$$

With V_a being the input voltage, A being the amplitude, the max voltage of the chirp signal, c being the chirp rate defined by the starting frequency subtracted from the ending frequency over the total run time, f_o being the starting frequency and t being the time. The amplitude is divided by 2 to prevent the motor from reversing direction.

The loop function is designed to run either the step trial functions or the chirp trials depending on whether the chirp function is defined within the code. If it isn't then the step trials are what is run by the Arduino. The data printed in the serial monitor section of the Arduino application is then saved as a excel file to allow MATLAB to read it and process the data.

Results and Discussion

The DC motor response was analyzed across 5 voltage values ranging from 4.8-9.6 Volts. Since the max voltage of the motor is 12 Volts, the motor was powered between 40-80% of its maximum capacity. The responses of trials 1-5 are graphed in figures 8-12. Each graph shows the RPM response over time after the motor is supplied with power. Graphed is the real-time measured RPM value in blue and the steady state RPM value in red, taking into consideration the oscillatory behavior of the measured values. The time constant of the system τ is the time it takes the motor to reach 63.2% of the steady state value. τ is defined by the black line, and the 63.2% value is denoted by the green line.

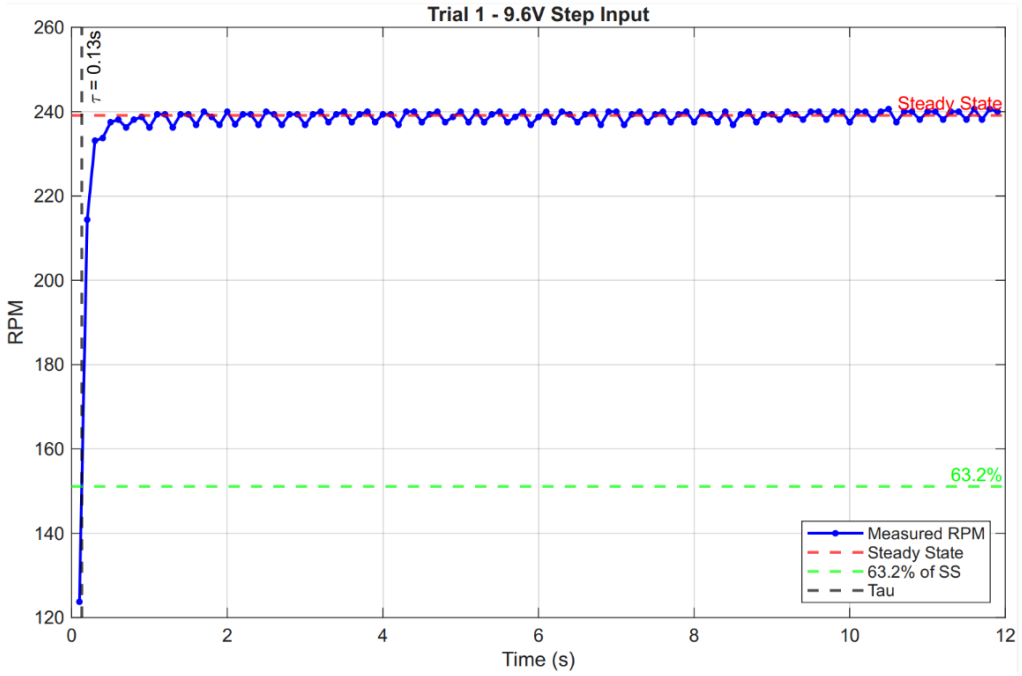


Figure 8. Trial 1 at 9.6V Step Input

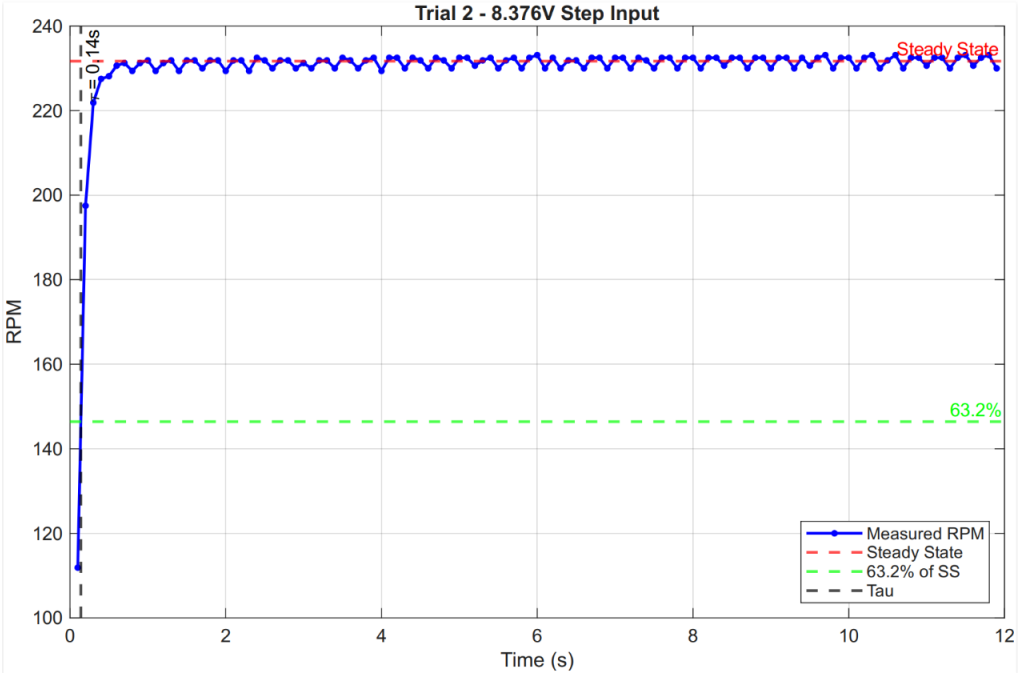


Figure 9. Trial 2 at 8.376V Step Input

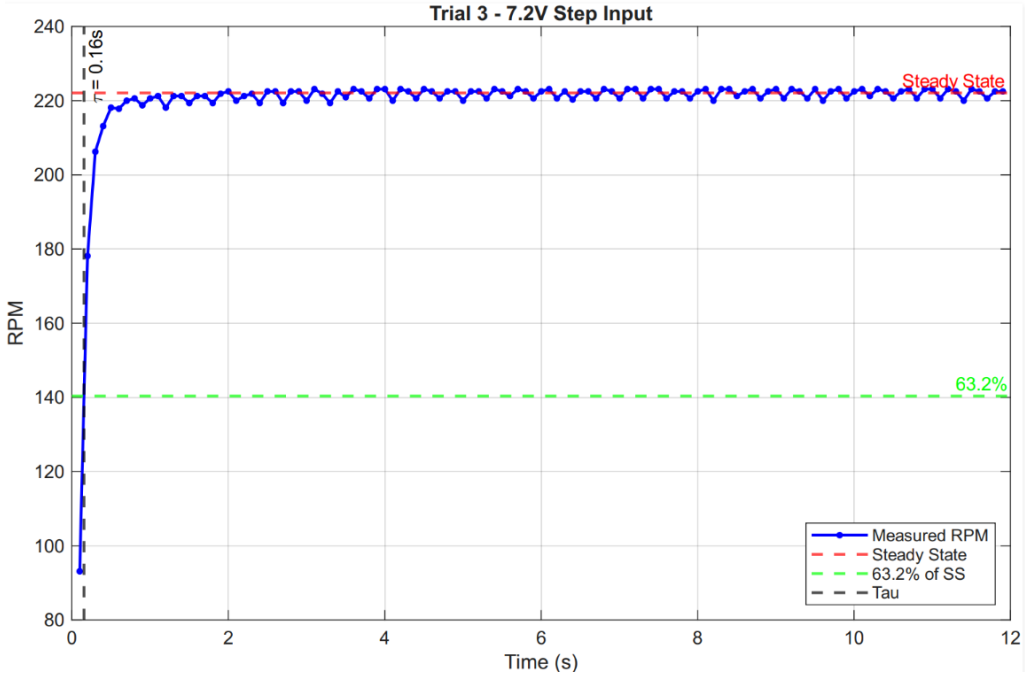


Figure 10. Trial 3 at 7.2V Step Input

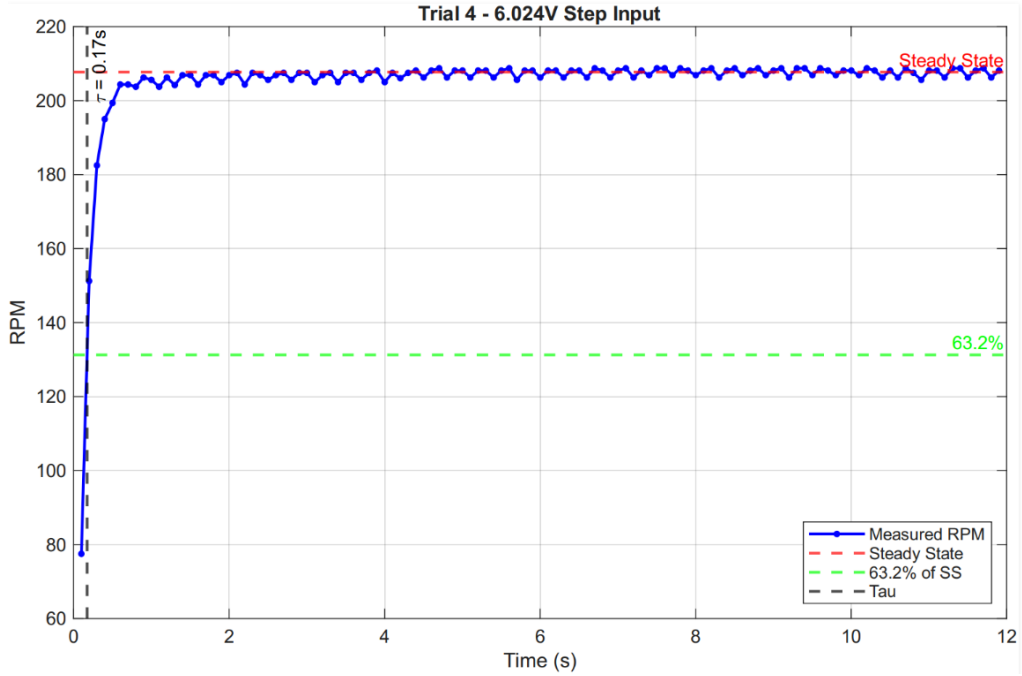


Figure 11. Trial 4 at 6.024V Step Input

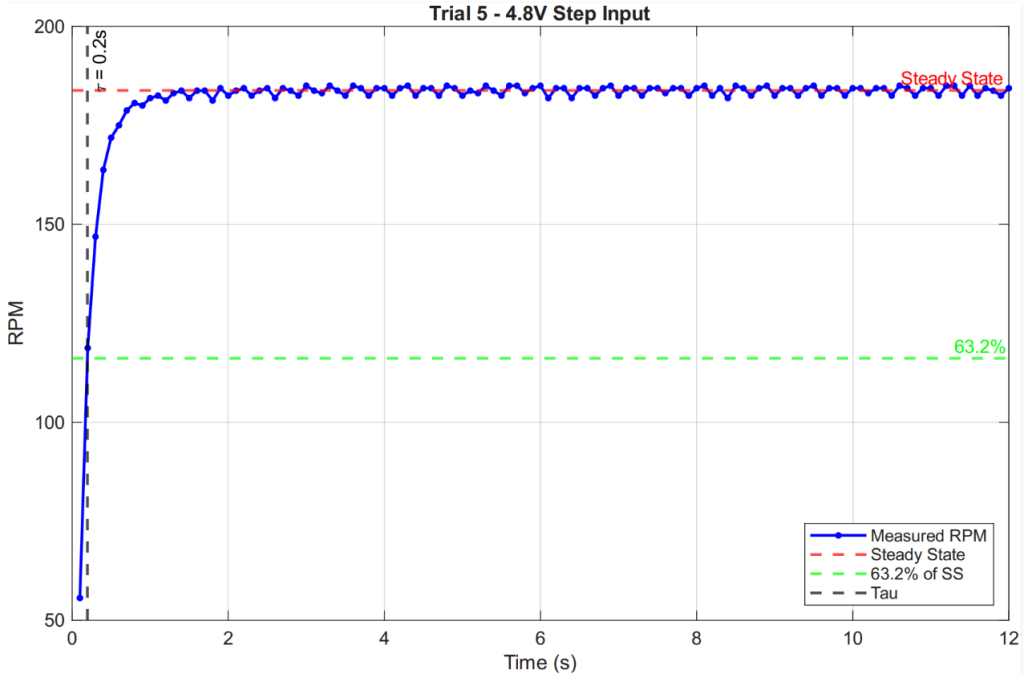


Figure 12. Trial 5 at 4.8V Step Input

Each of the five trials show similar trends at various voltage step inputs. From the graphs, both the measured and steady state RPM values have a positive correlation with increasing input voltages. For instance, at the maximum input voltage of 9.6V, the steady state RPM was 239 and at the lowest input voltage of 4.8V, the steady state RPM was 180. The step input response data closely resembled a first order system. This is consistent with the analytical RPM transfer function.

For each trial, an increase in input voltage led to a decrease in the time constant. For instance, at the highest voltage of 9.6Vs, the time constant was 0.13s and at the lowest voltage of 4.8V, the time constant was 0.2s. This indicates that the motor responds faster at higher input voltages because it takes less time to reach the 63.2% threshold. Physically, this can be explained by the fact that at the same resistance, a higher voltage leads to a higher current in the motor. This larger current results in greater torque in the motor, which leads to a higher acceleration of the output shaft.

To further validate the identified transfer function, a chirp input signal was applied to the motor. Figure 13 shows how the chirp signal was compared to a predicted response generated by the first order transfer function.

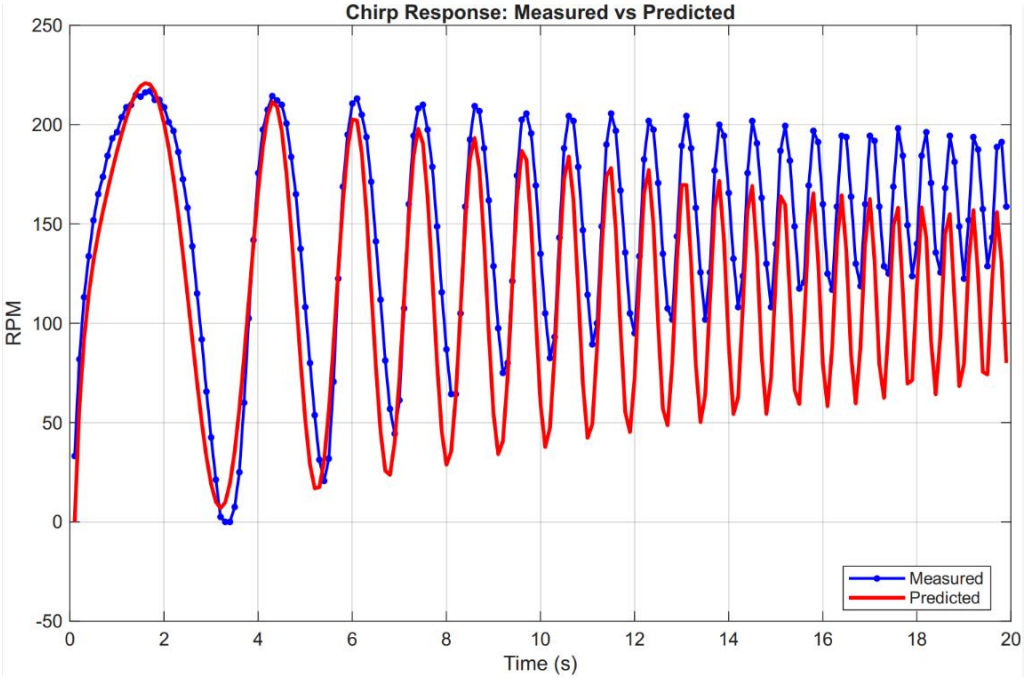


Figure 13. Comparison of Measured to Predicted Chirp Response

In plotting RPM over time, a blue line was used for the measured chirp response, and a red line was used for the predicted chirp response. Based on the results of the graph, the predicted model seems to hold well at lower frequencies, in the first 6 seconds, but over time as the frequency increases, the difference grows very noticeably. The graph shows that the amplitude of the RPM oscillations is greater in the predictive model than the physical measured model. There is also a small but noticable phase lag between the measured response and the predicted chirp response. Finally the RPM peak and average RPM values are far higher in the measured response than the predicted response. This all suggests that while the first order system might work well as lower frequences, it starts to break down over time at lower frequencies and becomes far less accurate.

Figure 14 shows a direct analysis of the error between the measured and predicted responses over the same time interval.

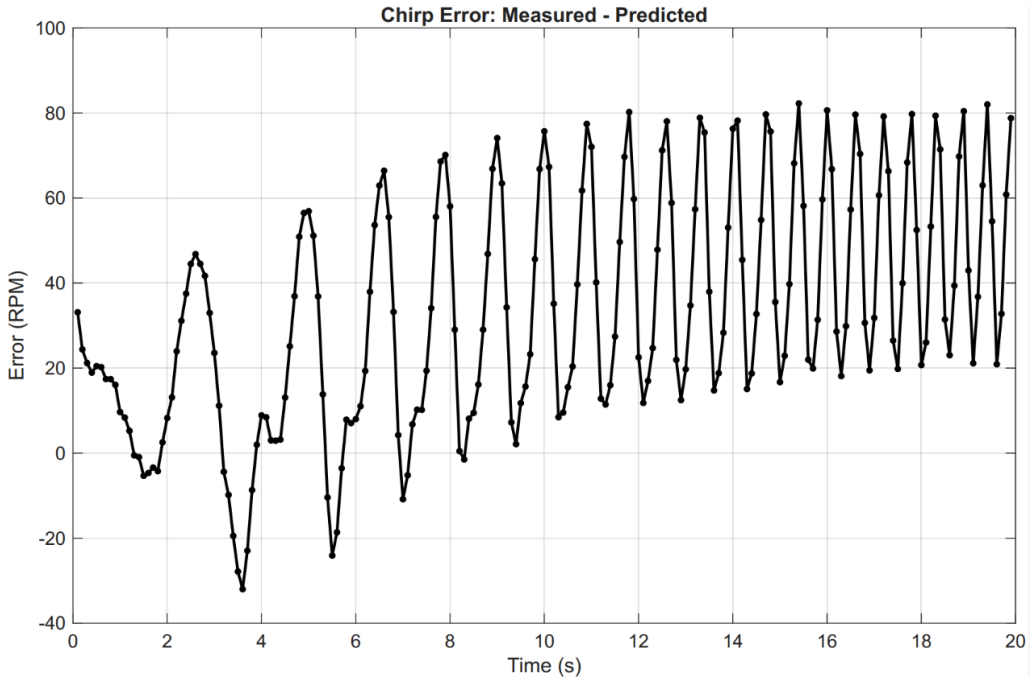


Figure 14. Chirp Error Between Measured and Predicted RPM

This graph shows the difference in RPM of the measured and predicted responses. This difference represents the error between the two models over time. The error values oscillate over time due to the phase gap between the two models. It can be seen in the graph that the error grows steadily over time and tends to taper off at a close to constant value at the end of the trial. The error growing over time indicates that the model is accurate for low frequency input where the dynamics are slower but becomes far less accurate at larger frequencies where the dynamics are faster. As previously mentioned, this suggests a breakdown of the first order transfer function used to model the system. Other sources that could contribute to the error are the model not accounting for other factors like friction and other losses, noise from the encoder, and inductance effects which were neglected for the first order approximation. A higher order system might be necessary to properly predict the behavior of the system at higher frequencies.

Conclusion

Throughout the comparison of predicted to experimental data there is close agreement between the two with disagreements coming as the frequency increases seen with the chirp trials. The identified transfer function shown in equation 2 gives a mean K value of 31.24 RPM/V and mean τ of .159 seconds.

$$G(s) = \frac{K}{\tau+1} = \frac{31.24}{.159+1} \quad (2)$$

The data was not completely stable with a standard deviation of around 5.32 RPM/V for the K value across the five step trials and for τ around .026s due to friction within the motor and back emf occurring within the circuit. The step response data displays first order system behavior which allows for the simplification of the derived transfer function seen above. The limitations of the first order system are present in the yielded error of the chirp trials with discrepancies as the frequency increases. Overall, our transfer function provides a reasonable model for a 12V, 300 RPM DC motor within an operated testing range.